

TinyCalc Language Specification

Compiler Front-End Documentation: Lexical & Syntax Contracts

1. Lexical Specification (Tokens)

The scanner (lexer) groups the raw source code characters into the following tokens. Whitespace and comments are consumed and discarded before being passed to the parser.

TOKEN NAME	REGEX / LITERAL	DESCRIPTION
Keywords		
PRINT	"print"	Output statement keyword
READ	"read"	Input statement keyword
Identifiers & Literals		
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	Variable names (e.g., x, result_1)
INTEGER	[0-9]+	Whole numbers
Operators		
ASSIGN	":="	Assignment operator
PLUS	"+"	Addition
MINUS	"-"	Subtraction and Unary Minus
MUL	"*"	Multiplication
DIV	"/"	Division
MOD	"%"	Modulo operation
POW	"^"	Exponentiation
Punctuation		

TOKEN NAME	REGEX / LITERAL	DESCRIPTION
LPAREN	"("	Left parenthesis
RPAREN	)"	Right parenthesis
SEMI	;"	Statement terminator
Ignored Tokens		
COMMENT	//.*	Single-line comment
WHITESPACE	[]+	Ignored spaces, tabs, and newlines

2. Grammar Specification (BNF)

This strict Backus-Naur Form (BNF) grammar enforces mathematical operator precedence and associativity, resolving ambiguity for the recursive-descent parser.

```

/* 1. Program Structure */
<program>      ::= <statement_list>

/* Left-recursive list to handle multiple statements */
<statement_list> ::= <statement>
                    | <statement_list> <statement>

/* 2. Statements */
<statement>     ::= <assignment_stmt>
                    | <print_stmt>
                    | <read_stmt>

<assignment_stmt> ::= IDENTIFIER ":=" <expr> ";"

<print_stmt>      ::= "print" <expr> ";"

<read_stmt>       ::= "read" IDENTIFIER ";"

/* 3. Expressions (Enforcing Precedence) */
/* Precedence Level 1: Left-associative addition & subtraction */
<expr>           ::= <term>
                    | <expr> "+" <term>
                    | <expr> "-" <term>

/* Precedence Level 2: Left-associative multiplication, division, modulo */
<term>           ::= <factor>
                    | <term> "*" <factor>
                    | <term> "/" <factor>
                    | <term> "%" <factor>

/* Precedence Level 3: Right-associative exponentiation */
<factor>         ::= <primary>
                    | <primary> "^" <factor>

/* Precedence Level 4: Base elements and unary minus */
<primary>        ::= "-" <primary>
                    | IDENTIFIER
                    | INTEGER
                    | "(" <expr> ")"

```